Assignment 1: Question 1

Course: Machine Learning
Student: Navneet Das
Date: 10 Aug 2026

Implementation of a Feedforward Neural Network for Classification

Objective

> Load the chosen classification dataset and explore its structure before modelling

Step 1: Load the Breast Cancer dataset.

In [80]:
```
# Importing necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
```

In [57]:
```
# Loading meta dataset
dataset = datasets.load_breast_cancer()
dataset.keys()
```

Out[57]:
```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename',
'data_module'])
```

In [58]:
```
# Cleaning dataset
data=pd.DataFrame(dataset["data"])
data["target"]=dataset["target"]
```

Step 2: Display the first five samples.

In [59]:
```
data.head()
```

Out[59]:

	0	1	2	3	4	5	6	7	8	9	...	21	22	
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1

5 rows × 31 columns

In [76]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   0       569 non-null    float64
 1   1       569 non-null    float64
 2   2       569 non-null    float64
 3   3       569 non-null    float64
 4   4       569 non-null    float64
 5   5       569 non-null    float64
 6   6       569 non-null    float64
 7   7       569 non-null    float64
 8   8       569 non-null    float64
 9   9       569 non-null    float64
 10  10      569 non-null    float64
 11  11      569 non-null    float64
 12  12      569 non-null    float64
 13  13      569 non-null    float64
 14  14      569 non-null    float64
 15  15      569 non-null    float64
 16  16      569 non-null    float64
 17  17      569 non-null    float64
 18  18      569 non-null    float64
 19  19      569 non-null    float64
 20  20      569 non-null    float64
 21  21      569 non-null    float64
 22  22      569 non-null    float64
 23  23      569 non-null    float64
 24  24      569 non-null    float64
 25  25      569 non-null    float64
 26  26      569 non-null    float64
 27  27      569 non-null    float64
 28  28      569 non-null    float64
 29  29      569 non-null    float64
 30  target  569 non-null    int32  
dtypes: float64(30), int32(1)
memory usage: 135.7 KB
```

In [77]:
```
data.describe()
```

Out[77]:

	0	1	2	3	4	5	6	
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.00000
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.04891
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.03880
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.00000
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.02031
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.03350
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.07400
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.20120

8 rows × 31 columns

Step 3: Display the dataset shape.

In [61]:
```
print(f"the shape of the given dataset is :{data.shape}")
```

```
the shape of the given dataset is :(569, 31)
```

Step 4: Display the number of features.

In [62]:
```
# separation of data
features=data.iloc[:,:-1]
target=data["target"]
```

In [63]:
```
print(f"The number od features are {features.shape[1]}")
```

```
The number od features are 30
```

Step 5: Display the number of classes.

In [71]:
```
print(f"in total we have {data["target"].nunique()} classes which are {data["target"].unique(
```

```
in total we have 2 classes which are [0 1] or ['malignant' 'benign']
```

Step 6: Check for missing values.

In [75]:
```
data.isnull().sum()
```

Out[75]:
```
0         0
1         0
2         0
3         0
4         0
5         0
6         0
7         0
8         0
9         0
10        0
11        0
12        0
13        0
14        0
15        0
16        0
17        0
18        0
19        0
20        0
21        0
22        0
23        0
24        0
25        0
26        0
27        0
28        0
29        0
target    0
dtype: int64
```

Question 2

Data Preprocessing

Objective

> Prepare the dataset for neural network training

Step 7: Split the dataset into training and testing sets.

In [79]:
```
xtrain,xtest,ytrain,ytest=train_test_split(data.iloc[:,:-1],data["target"],test_size=0.2,shuf
print(f"shape of xtrain :{xtrain.shape}")
print(f"shape of xtest :{xtest.shape}")
print(f"shape of ytrain :{ytrain.shape}")
print(f"shape of ytest :{ytest.shape}")
```

```
shape of xtrain :(455, 30)
shape of xtest :(114, 30)
shape of ytrain :(455,)
shape of ytest :(114,)
```

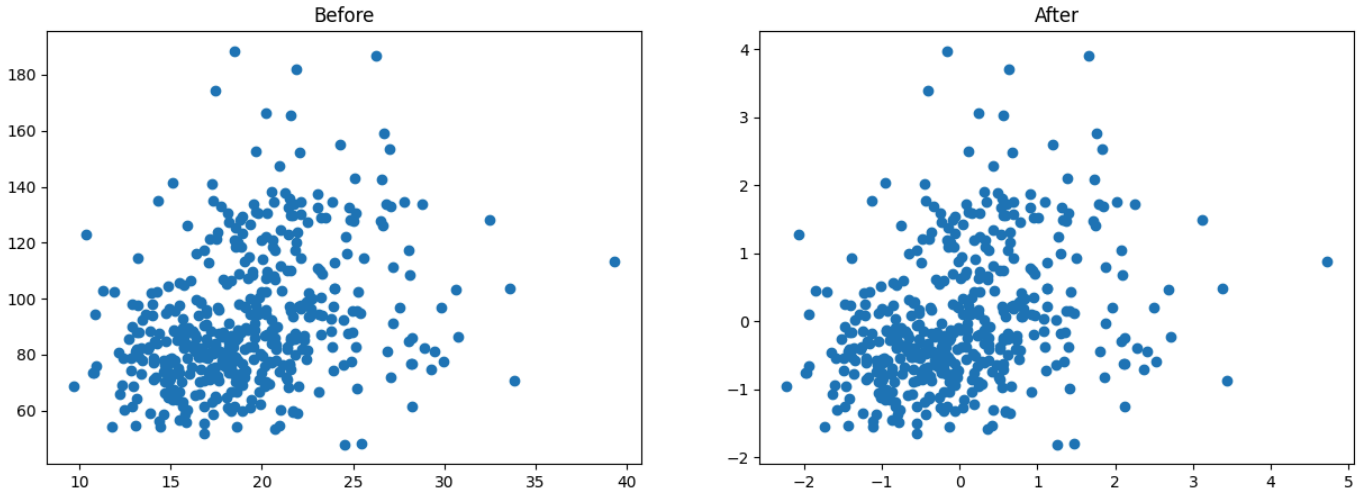
Step 8: Standardize all numerical features.

In [81]:
```
ss = StandardScaler()
xtrain_scaled = ss.fit_transform(xtrain)
xtest_scaled = ss.transform(xtest)
xtrain_scaled
```

Out[81]:
```
array([[-1.44075296, -0.43531947, -1.36208497, ...,  0.9320124 ,
         2.09724217,  1.88645014],
       [ 1.97409619,  1.73302577,  2.09167167, ...,  2.6989469 ,
         1.89116053,  2.49783848],
       [-1.39998202, -1.24962228, -1.34520926, ..., -0.97023893,
         0.59760192,  0.0578942 ],
       ...,
       [ 0.04880192, -0.55500086, -0.06512547, ..., -1.23903365,
        -0.70863864, -1.27145475],
       [-0.03896885,  0.10207345, -0.03137406, ...,  1.05001236,
         0.43432185,  1.21336207],
       [-0.54860557,  0.31327591, -0.60350155, ..., -0.61102866,
        -0.3345212 , -0.84628745]])
```

In [100…
```
fig ,(ax1 , ax2) = plt.subplots(nrows=1,ncols=2, figsize=(15,5))
ax1.scatter(xtrain[1],xtrain[2])
ax1.set_title("Before")
ax2.scatter(xtrain_scaled[:,1],xtrain_scaled[:,2])
ax2.set_title("After")
```

Out[100…
```
Text(0.5, 1.0, 'After')
```



Step 9: Explain why feature scaling is important for neural networks.

- **Prevents Dominance:** *Large-scale inputs drown out small-scale inputs during early training updates.*
- **Fair Learning** *Every input feature gets an equal chance to influence the network's weights.*